

Work with data using Spark SQL

7 minutes

The Dataframe API is part of a Spark library named Spark SQL, which enables data analysts to use SQL expressions to query and manipulate data.

Creating database objects in the Spark catalog

The Spark catalog is a metastore for relational data objects such as views and tables. The Spark runtime can use the catalog to seamlessly integrate code written in any Spark-supported language with SQL expressions that may be more natural to some data analysts or developers.

One of the simplest ways to make data in a dataframe available for querying in the Spark catalog is to create a temporary view, as shown in the following code example:

Python

```
df.createOrReplaceTempView("products_view")
```

A *view* is temporary, meaning that it's automatically deleted at the end of the current session. You can also create *tables* that are persisted in the catalog to define a database that can be queried using Spark SQL.

Tables are metadata structures that store their underlying data in the storage location associated with the catalog. In Microsoft Fabric, data for *managed* tables is stored in the **Tables** storage location shown in your data lake, and any tables created using Spark are listed there.

You can create an empty table by using the `spark.catalog.createTable` method, or you can save a dataframe as a table by using its `saveAsTable` method. Deleting a managed table also deletes its underlying data.

For example, the following code saves a dataframe as a new table named **products**:

Python

```
df.write.format("delta").saveAsTable("products")
```

📌 Note

The Spark catalog supports tables based on files in various formats. The preferred format in Microsoft Fabric is **delta**, which is the format for a relational data technology on Spark named *Delta Lake*. Delta tables support features commonly found in relational database systems, including transactions, versioning, and support for streaming data.

Additionally, you can create *external* tables by using the `spark.catalog.createExternalTable` method. External tables define metadata in the catalog but get their underlying data from an external storage location; typically a folder in the **Files** storage area of a lakehouse. Deleting an external table doesn't delete the underlying data.

💡 Tip

You can apply the same partitioning technique to delta lake tables as discussed for parquet files in the previous unit. Partitioning tables can result in better performance when querying them.

Using the Spark SQL API to query data

You can use the Spark SQL API in code written in any language to query data in the catalog. For example, the following PySpark code uses a SQL query to return data from the **products** table as a dataframe.

Python

```
bikes_df = spark.sql("SELECT ProductID, ProductName, ListPrice \n                      FROM products \n                      WHERE Category IN ('Mountain Bikes', 'Road Bikes')")\n\ndisplay(bikes_df)
```

The results from the code example would look similar to the following table:

 Expand table

ProductID	ProductName	ListPrice
771	Mountain-100 Silver, 38	3399.9900
839	Road-750 Black, 52	539.9900
...	...	...

Using SQL code

The previous example demonstrated how to use the Spark SQL API to embed SQL expressions in Spark code. In a notebook, you can also use the `%%sql` magic to run SQL code that queries objects in the catalog, like this:

SQL

```
%%sql

SELECT Category, COUNT(ProductID) AS ProductCount
FROM products
GROUP BY Category
ORDER BY Category
```

The SQL code example returns a resultset that is automatically displayed in the notebook as a table:

 Expand table

Category	ProductCount
Bib-Shorts	3

Category	ProductCount
Bike Racks	1
Bike Stands	1
...	...

Next unit: Visualize data in a Spark notebook

[< Previous](#)[Next >](#)